# 4D-STEM Software Proposal and Requirements Specification

Ciaran Welsh / SEM Team

September 2, 2025


## Contents

# 1 Introduction

This document is a proposal for how to move forward in software for 4D-stem at ASI. Its intention is to be implementation agnostic although we already have an idea on how this might look and a few high level implementation details have crept in. Should we choose to implement the tools and features outlined in this document, then this document would serve as a reference for software developers to know *what* to build, a reference for application engineers to know *what* to test and a reference for sales force to know *what* to promise prospective clients. It should also describe to everybody *why* we are building the software in the first place.

Although we have tried to be comprehensive, we do not expect to capture everything that a 4D-stem software package needs up front. Instead, this document will be updated with relevant details when it is appropriate to do so.

If accepted, this proposal will be split into small managable chunks of work that do not exceed 4 weeks of development at a time. Features will be tested in-house and given to customers for testing and feedback. This is not a waterfall project where in a years time we give a customer some software that they do not like or doesn't fit their needs.

## 1.1 Background

4D-STEM (Four Dimensional Scanning Transmission Electron Microscopy) is a microscopy technique where a focused electron beam is scanned across a sample, producing a 2D diffraction pattern at each position. The resulting four-dimensional dataset, with two spatial and two reciprocal-space coordinates, enables detailed analysis of structure, strain, and other material properties.

The advantage of Timepix3 and Timepix4 technology lies in their event-driven acquisition. Operating in event-driven mode allows for significantly faster scanning and removes the concept of a traditional shutter (when in continuous acquisition mode). Event-based data is naturally well suited for representing sparse matrices and handling low-dose experiments, which is an increasingly important trend and often a key limitation in 4D-STEM.

From a business perspective, the primary objective is to increase ASI detector sales by embedding them within a complete 4D-STEM solution. The addition of 4D-STEM capability substantially enhances the value proposition of the detector, making this initiative a strong growth opportunity for ASI. Our core goal is to deliver a 4D-STEM solution that meets key user requirements both online and offline; including essential functionality for data acquisition and interoperativity with other 4d-stem processing tools.

From the outset, the design will incorporate support for the next generation of Timepix (Timepix4). While full-rate Timepix4 support may not be available immediately, the system will be developed with modularity in mind. This will allow us to integrate higher-performance components in the future, such as GPU-accelerated modules.

## 1.2 Positioning in terms of existing software

At a super high level, our software system needs to control Tpx3, Mpx3 and Tpx4 detectors and receive data produced by them for further analysis. We also need to assist in determining whether a region of sample is interesting enough for further analysis. This already necessitates several diverse features ranging from low level detector control to a live preview of the detector image. In the more distant past, Sophie and Dexter were our main software offerings. These were "monolithic" (single deployable unit) applcations that took on all responsibilities needed for all the requirements. Whilst the exact reasons for the deprecation of these tools were not recorded in any detail, they had significant enough drawbacks to warrent a redesign of the architecture into a distributed system (multiple deployable units), which are now known as Serval and Accos. To pre-load the discussion below, the proposal in this document is to split Serval functionality further into another distributed unit to handle 4D-stem data processing and to replace the presentation layer with another unit specific to 4D-STEM.

To elaborate, when Serval/Accos were designed the focus was on core detector control and acquisition. We did not consider application specific/focused areas. Since then, our organisation has moved to what we are calling "application focused" teams, whereby 4D-STEM is one of our application focused areas. The proposal

here is to have our software directly miorror the organisation of our company by making the architecture and product reflect the team boundaries as well as how we present products in marketing. A dedicated 4D-STEM application aligns with our 4D-STEM team and clarifies our market positioning as a 4D-STEM solutions company.

To date, 4D-STEM pipelines prototyped inside Serval are unfinished and have performance, testing, and stability issues. We could finish them in Serval (pros: reuse and a simpler "one Serval" to rule them all), but the cons are significant: 4D-STEM changes interfere with core acquisition, shared releases slow delivery, and the 4D-STEM product is obscured under Serval/Accos.

Since the development of Serval and Accos, we have invested time in a software package called "Luna" dedicated to the processing of Timepix3 and Timepix4 data. Luna is written in rust which is renowned both for its picky compiler and memory safety without garbage collection; making it an excellant choice when the need is high performance online data processing.

In the following sections, we describe the system we intend to build from a high-level perspective. We describe the environment the system needs to run, including software and hardware dependencies, and we outline the needs of the end user that this software aims to solve. We outline functional requirements and the capabilities needed for the software to succeed.

# 2 Overall Description

The purpose of this software is to provide a fully integrated workstation for controlling detectors and scan generators so scientists can run 4D-STEM acquisitions easily. The scope is limited to acquisition and the minimal real-time and offline processing required to decide whether a sample region merits further analysis.

A high-performance live image stream is essential to give microscopists timely feedback while navigating regions of interest. Because "further analysis" is explicitly out of scope, we need clear handoff boundaries where users can move to other software. That is, in order to integrate ourselves into the 4D-Stem software ecosystem, we need to properly interoperate with downstream analysis tools, including LiberTEM, py4DStem and HyperSpy.

## 2.1 User Classes and Characteristics

The primary users are 4D-STEM scientists, who also operate the microscope. Users range from computationally literate power users to those without much experience. Both users are mainly focused on scientific outcome rather than on the processing required to get them there. The software abstracts data acquisition and visualisation details so that no programming skills are required for them to do just that.

## 2.2 Operating Environment

The application supports Windows, Ubuntu, and macOS at launch; other Linux distributions can be built on request. Our primary system is ubuntu and several aspects of our tooling are already optimized for ubuntu. We therefore aim for performance targets primarily on ubuntu, and only secondarily on the other operating systems. The software shall run on a high-performance lab workstations with specifications sufficient for sustained high-rate data capture and processing. GPU acceleration is planned for future versions but is not required initially. The system operates on localhost only, with all components running on the same machine. Storage must sustain the highest practical write speeds for the selected output format.

## 2.3 User Needs

1. **Full Integration with Serval for detector control whilst being exposed to only what they need**
   *Rationale*: Serval must be used for detector control but minimizing the user interaction with Serval reduces the learning curve and enhances usability.

2. **Real-time visual feedback during scans**
   *Rationale*: Enables rapid navigation to regions of interest and immediate quality assessment. Also prevents wasted beam time.

3. **Ability to run essential offline analyses inside the application, such as offline image reconstruction methods.**
   *Rationale*: Enables quality assessment decisions without exporting large datasets to external tools.

4. **Acquire continuous or multi-frame ('Z-stack') scans**
   *Rationale*: Lets users monitor radiation damage frame-by-frame and perform statistics based on multiple measurements in a region.

5. **Flexible detector masking (hardware or software) to select pixels for subsequent analysis.**
   *Rationale*: Enables selection of diffraction bands for online and offline image reconstruction algorithms.

6. **Switchable scan modes to balance performance and user feedback trade-offs**
   *Rationale*: Allocation of computational resources should be allocated differently depending on whether the user is searching for interesting features or whether they have already found them and want to acquire data for further analysis.

7. **Export raw *or* processed data in open formats (e.g. SparseArray, HDF5, Zarr) and where appropriate, image data**
   *Rationale*: Guarantees images can be saved and interoperability with downstream analysis suites such as LiberTEM, HyperSpy, and py4DSTEM.

8. **Real-time observability of acquisition-pipeline health (hit-rate, processing load, synchronization)**
   *Rationale*: Gives user vital feedback that can be used to understand what is happening in the pipeline. This includes amounts of dropped data, per chip level hit rates and data throughput/latency rates.

9. **Dedicated 4D-STEM interface separate from the generic detector UI**
   *Rationale*: Build with a 4D-stem first mentality. Makes software 4d-stem centric, minimises cognitive load on our users needing to compete with more general use cases of the timepix3 detector and prevents accidental changes to routine detector operations. Separation of concerns.

10. **Immediate and clearly organized access to scan-generator controls**
    *Rationale*: Scan controls are some of the most important in 4D-STEM, they should be prevalent and obvious in the user workflow for better user experience, operating on the principle of least clicks to control your scan.

## 2.4 Definitions and Acronyms

**CBED: Convergent Beam Electron Diffraction**

**1.** (a) A convergent (focused or defocused) electron probe produces a diffraction disk in the reciprocal plane, where the detector is positioned. This forms the basis of Convergent Beam Electron Diffraction (CBED), in which the disk size in reciprocal space is inversely related to the probe size in real space.

**2. vADF: virtual Annular Dark Field**

(a) The user defines a ring in the detector's dark-field region (beyond the probe's convergence angle), which serves as a discriminator for dark-field contrast. This contrast is roughly proportional to the atomic number (Z) of the material and the proximity of the electron probe to the atom. As a rule of thumb, heavier atoms scatter electrons to larger angles, so positioning the ring further from the optical axis enhances their contrast.

3. **iDPC: integrated Differential Phase Contrast**

   (a) Contrast is generated by integrating the differential phase signal across segmented regions of the detector. Instead of relying on high-angle scattering, iDPC is sensitive to the phase shifts induced by variations in the electrostatic potential of the sample. This makes the technique particularly well-suited for visualizing light elements and weakly scattering materials. The rule of thumb is that iDPC provides nearly linear contrast with respect to the projected potential, enabling both heavy and light atoms to be visualized simultaneously with high fidelity.

4. **iCOM: integrated Center Of Mass**

   (a) In iCOM imaging, contrast is generated by computing the center-of-mass of the transmitted electron intensity across the detector. This approach directly maps the local momentum transfer from the electron beam to the sample, providing sensitivity to both electric and magnetic fields as well as structural variations. Unlike high-angle scattering techniques, iCOM captures subtle deflections of the probe, making it particularly effective for visualizing light elements and weak phase objects. The rule of thumb is that the magnitude and direction of the center-of-mass shift correspond to the strength and orientation of the local projected fields, enabling quantitative interpretation of sample properties.

5. **Diffraction:**

   (a) The concept of diffraction must be understood in relation to both the sample statistics and the characteristics of the electron beam interacting with the sample:

      i. **Parallel (collimated electron beam):** When using a large parallel beam that illuminates many unit cells of a crystalline material, sharp diffraction spots appear, corresponding to the Bragg reflections of the unit cell. In the case of a polycrystalline material, these spots are replaced by diffraction rings, as multiple grain orientations contribute to the same reflection.

      ii. **Convergent beam (CBED):** As the beam is focused to cover fewer unit cells (a smaller "kernel" in real space), diffraction spots broaden into disks.

   (b) From a quantum mechanical perspective, this can be linked to Heisenberg's uncertainty principle: greater certainty in position (real space, where the localized probe interacts with the sample) leads to reduced certainty in momentum (reciprocal space), blurring the Bragg reflections due to limited sampling statistics. And the opposite is true for the case of Parallel illumination

# 3 System Features and Requirements

## 3.1 Functional Requirements

### 3.1.1 Online Image Reconstruction

1. Must **REQ-OIR-01 – Real-time vADF reconstruction.** Raw event data received from Serval via TCP shall be processed in real time into images using the vADF algorithm. Processing is performed by *luna-stem*/*luna-core* and displayed in the 4D-STEM UI. For selection of pixels that contribute to final vADF image, an annular ROI shall be used (item 16).

2. Must **REQ-OIR-02 – Preview non-interference.** Preview must not reduce performance to such a point that data integrity is compromised.

3. Could **REQ-OIR-03 – GPU-accelerated vADF.** Provide a GPU-accelerated implementation of vADF on supported hardware. Especially when targeting tpx4 (in future) which has x5 the performance requirements of a tpx3 turbo, a GPU backend should be built.

### 3.1.2 Offline Image Reconstruction

1. Must **REQ-OFR-01 – Single CBED display.** When a user selects a probe position, the system shall display the corresponding CBED diffraction pattern from disk.

2. Must **REQ-OFR-02 – PACBED (Position Averaged Convergent Beam Electron Diffraction).** The system shall compute and display the PACBED over a selected ROI of probe positions in the scan (or all of them).

3. Must **REQ-OFR-03 – Offline vADF.** The system shall recompute vADF from stored data by summing intensities within the selected detector region for each probe position. The selection ROI shall be annular, but not necesarily centered in the detector (As the CBED may be shifted).

4. Must **REQ-OFR-04 – iCOM / iDPC.** Provide offline iCOM / iDPC reconstruction on supported datasets. Performance must be "reasonable" but not heavy duty, like the online algorithms

5. Must **REQ-OFR-05 – Asynchronous execution + progress.** Offline operations shall run asynchronously with progress reporting and possibility of user cancellation. Partial outputs are marked incomplete with a warning and message.

6. Could **REQ-OFR-06 – Random-access readiness.** If needed for performance, maintain an index that maps probe position to data blocks/offsets next to processed data output to facilitate fast lookup and retrieval of data for building images (such as recomputing CBED).

7. Won't **REQ-OFR-07 – Ptychography (offline).** Offline ptychography is out of scope.

8. Won't **REQ-OFR-08 – 4D-STEM EELS (offline).** Offline 4D-STEM EELS is out of scope.

### 3.1.3 Serval Integration

1. Must **REQ-SRV-01 – Connect or spawn Serval.** On startup, the system shall either (a) connect to a reachable Serval instance at the configured host:port, or (b) spawn Serval as a child process whose lifetime is tied to the 4D-STEM UI.

2. Must **REQ-SRV-02 – Version negotiation.** On initial connect, the system shall retrieve Serval's API version and compare it to the supported range. If out of range, the UI shall warn the user.

3. Must **REQ-SRV-03 – State subscription (push).** The client shall subscribe to Serval state changes (e.g., detector {Disconnected, Connected, Idle, Running}). Won't The system shall not use periodic polling to keep up to date with serval connectivity status.

4. Must **REQ-SRV-04 – Detector connection sequence.** Upon receiving *detector_connected* signal, the system shall: (1) upload DACs, (2) upload pixel configuration (BPC), (3) set data destination, (4) load saved detector settings (e.g. bias) for this detector chipboard Id. Failures stop the sequence and surface a recoverable error with instructions on how to resolve the issue. Dacs and BPC are produced by equalisation in Accos and provided when a user purchases a system from us.

5. Must **REQ-SRV-05 – Serval Command error model.** All Serval command failures shall be presented to the user as human readable message and hint on how to resolve the error.

6. Must **REQ-SRV-06 – Connectivity loss during acquisition.** If the control connection to Serval drops during an active acquisition the acqiusition is considered invalid. The system shall detect connection loss and abort the acquisition gracefully.

7. Must **REQ-SRV-07 – Logging and correlation.** Every Serval command, response, state change, and error shall be logged in a serval http log stored per workspace

8. Must **REQ-SRV-08 – Transport keepalives.** The client shall use protocol keepalives to detect dead connections without polling for state.

9. Must **REQ-SRV-09 – Endpoint configuration.** Serval host, port, and transport addresses shall be configurable in the UI; changes take effect on "accept" button press. Sensible defaults are pre-selected.

10. Won't **REQ-SRV-10 – Equalization in client.** The client shall not perform detector equalization; that function remains in Accos.

11. Won't **REQ-SRV-11 – Detector orientation in client.** The client shall not manage detector orientation. It is complicated to implement and users can rotate images after the fact in any basic image processing tool.

#### 3.1.4 Z-stacking Frames

1. Should **REQ-ZST-01 – Fixed-length Z stack.** When the user sets $Z > 1$ and presses *Start (Acquire)*, the system shall execute exactly $Z$ full probe-grid scans. Every detector event shall be assigned to frame and probe position.

2. Should **REQ-ZST-02 – Continuous Z mode.** When Continuous Z mode is selected, the system shall repeat full probe-grid scans until the user issues *Stop* or an automatic stop condition occurs (e.g., disk guard). Frames in continuous mode shall use a monotonically increasing *frame_id* $\in \{0, 1, 2, \dots\}$.

3. Should **REQ-ZST-03 – Frame output writing.** Frame data shall be written to disk in the user-selected format(s).

#### 3.1.5 Output and Raw Data Collection

1. Must **REQ-OUT-01 – Raw data immutability.** Raw event data shall be written to disk exactly as received from Serval (ordering and content preserved), with no corrections applied in-line by the client or Serval.

2. Must **REQ-OUT-02 – Corrections policy.** Corrections required for scientific correctness (e.g., tram-lane and PLL corrections) shall be applied in reconstructions (online/offline), but not to raw data.

3. Won't **REQ-OUT-03 – Timewalk correction.** Timewalk correction shall not be applied. The time resolution required in 4D-stem is usually not small enough to be impacted by Timewalk effects.

4. Must **REQ-OUT-04 – Offline sparse data construction.** Offline conversion of raw tpx3 binary format to LiberTEMs sparse array format.

5. Should **REQ-OUT-05 – Online sparse data construction.** If performance allows, online processing to sparse arrays would be nice.

6. Should **REQ-OUT-06 – Zarr export.** Offline conversion of raw binary tpx3 data to Zarr format

7. Should **REQ-OUT-07 – HyperSpy HDF5 export.** Offline conversion of raw binary tpx3 data to HyperSpy HDF5 format

8. Must **REQ-OUT-08 – Image export (2D).** Any 2D image produced (e.g., vADF, CBED, iCOM maps) shall be exportable to TIFF and PNG; 16-bit TIFF supported; metadata embedded where format allows.

9. Could **REQ-OUT-09 – ROI-based export.** For data reduction, users may export subsets (ROIs in real space or diffraction space) to supported formats without duplicating full datasets.

10. Must **REQ-OUT-10 – Directory layout and naming.** Acquisitions shall use a standardized directory layout and naming convention. Users can modify their pattern with a selection of template strings. Datetime, run ID, sample ID are examples.

11. Could **REQ-OUT-11 – Compression and chunking.** For formats supporting it (Zarr/HDF5), chunking and compression shall be implemented for offline use.

#### 3.1.6 Pixel Masking

1. Must **REQ-PMK-01 – Software masking.** The system shall support software inclusion zones or 'mask' defined on the detector pixel grid (rectangular, circular, and annular at minimum). Software masks are applied in image reconstruction; raw data remain unchanged.

2. Could **REQ-PMK-02 – Hardware masking.** The system shall support hardware masking by uploading a pixel configuration via Serval.

3. Must **REQ-PMK-03 – Mask mode selection.** If both hardware and software masking are available, the UI shall allow selecting the active mode; switching is permitted only when idle.

4. Must **REQ-PMK-04 – Mask visualization.** Masks shall be visibly overlaid on the viewport; hardware and software masks use distinct visual styles.

5. Must **REQ-PMK-05 – Autoload masks.** When loading an existing dataset/measurement, previously saved masks shall autoload and appear in the UI.

#### 3.1.7 Acquisition Modes

1. Must **REQ-ACQ-01 – Search mode.** Search mode shall render real-time preview (vADF) and, by default, shall not persist raw events or reconstructions.

2. Must **REQ-ACQ-02 – Acquire mode (Raw).** Acquire mode shall persist raw data data to disk in tpx3 binary data

3. Could **REQ-ACQ-03 – Acquire mode (Sparse).** If performance allows, data should be stored to disk in directly in sparse format.

4. Must **REQ-ACQ-04 – Mode transitions.** Mode selection shall be explicit and visible; mode changes are blocked while an acquisition is active.

5. Should **REQ-ACQ-05 – Parameter locking in Acquire.** On *Start (Acquire)*, scan parameters and masks are locked for the duration of the run.

#### 3.1.8 Scan Generator

1. Must **REQ-SCN-01 – Point electronic first.** The principle scan generator supported shall be the one from Point Electronic that we have already been working with.

2. Should **REQ-SCN-02 – Other scan generators.** Other scan generators shall be supported as needed

3. Must **REQ-SCN-03 – Backend selection.** The system shall support selecting a scan-generator backend from a configured list; unavailable backends appear disabled.

4. Must **REQ-SCN-04 – Start-time synchronization.** Detector acquisition start and scan start shall be synchronized at the start and end of scan boundary.

5. Could **REQ-SCN-05 – ROI/geometry translation.** As "sugar" on the top, the scan generator shall accept ROI-derived geometries and produce matching scan grids.

6. Must **REQ-SCN-06 – Graceful stop.** On *Stop*, the scan generator shall honor the 'stop now' and 'stop at frame boundary' semantics from Z-stacking.

#### 3.1.9 Observability and Pipeline Health

1. Must **REQ-OBS-01 – Metric definitions.** Performance metrics (hit-rate, throughput in bits/s,latency) shall be computed and prefented to the user, updated at regular intevals.
2. Must **REQ-OBS-02 – Statistic granularity.** Statistics shall be computed for the whole detector and per-chip.
3. Must **REQ-OBS-03 – Observability overhead bound.** Observability should not interfere with processing by introducing significant overhead
4. Could **REQ-OBS-04 – Scan detector sync metric.** Provide a sync metric estimating jitter between scan-box timing and detector hit timestamps
5. Must **REQ-OBS-05 – Dropped-data visibility.** Dropped/lost/mis-mapped events shall be tracked and displayed to the user in UI.

#### 3.1.10 User Interface GUI

This section describes the UI counterpart of the feature only. Every UI component is backed by a non-UI component and its business logic is tested away from the UI in the form of unit/integration/end-to-end testing.

1. Must **REQ-UI-01 – Standalone launch.** The application shall launch as a standalone process when the user double-clicks the 4D-STEM UI icon or selects it from the native OS menu.
2. Must **REQ-UI-02 – Restricted detector controls.** The UI shall display only the detector controls listed that they need for 4D-STEM acquisition. It may be that we can abstract *all* detector controls away from the user.
3. Must **REQ-UI-03 – Derived detector settings.** Where possible, detector settings shall be derived automatically from scan parameters without additional user interaction. If the setting can be abstracted from the user, then it should be.
4. Must **REQ-UI-04 – Scan engine Configuration.** The UI shall present a control window to configure the available scan engine. The scan engine must be supported by software.
5. Must **REQ-UI-05 – Scan geometry controls.** The UI shall present controls for geometry including: scan height and width (in probe positions), prescan x/y, always visible.
6. Must **REQ-UI-06 – Dwell control.** The UI shall present a dwell-time control, always visible.
7. Must **REQ-UI-07 – Z-frames control.** The UI shall present a control for the number of frames $Z$ (frame = one full scan of $H \times W$), always visible.
8. Must **REQ-UI-08 – Start (Preview).** The UI shall provide a Start (Search/Preview) control.
9. Must **REQ-UI-09 – Start (Acquire).** The UI shall provide a Start (Acquire) control.
10. Must **REQ-UI-10 – Output Location.** The UI presents a widget to collect output location. The widget accepts text input and selection from the native filesystem widget
11. Must **REQ-UI-11 – Output format.** A dropdown containing supported output formats is presented and always visible.
12. Must **REQ-UI-12 – Workspaces.** Users shall be able to create/open a workspace; a workspace can contain multiple measurements.
13. Must **REQ-UI-13 – Measurement metadata.** Each measurement shall have metadata persisted as JSON (schema defined in Data Formats), accessible via the UI but hidden by default. All scan and detector parameters are included in the measurement metadata plus comments written by users themselves.

14. Must **REQ-UI-14 – Stop button.** The UI shall provide a Stop control that is enabled only while Preview or Acquire is running.

15. Must **REQ-UI-15 – Stop semantics.** When the user presses *Stop*, the system shall finish the current probe position and then stop. A preference 'Stop at frame boundary' shall, when enabled, finish the current frame before stopping.

16. Must **REQ-UI-16 – ROIs.** The user shall be able to create, update/move and remove rectangular, circular or annular ROIs by click-and-drag on the viewport.

17. Must **REQ-UI-17 – ROI to scan geometry.** The system shall translate a rectangular ROI into an equivalent scan geometry. ROIs snap to the pixel grid so values derived are integers.

18. Must **REQ-UI-18 – ROI for diffraction views.** The system shall allow using ROI selections to build diffraction images (e.g. averaged CBED of the selection).

19. Could **REQ-UI-19 – ROI expanded view.** A ROI shall be expandable via invoking the 'right click ¿ Expand ROI' function. The viewport resizes to zoom in making the boundary of the ROI the new edges of the imae.

20. Could **REQ-UI-20 – Dockable controls.** Control panels shall be dockable/undockable and toggleable via the View menu. A default layout is exposed for return to factory settings.

21. Must **REQ-UI-21 – Persistence.** UI settings (layout, last workspace, panel visibility, scan control values) shall persist across sessions and at the granularity of a workspace. Detector specific controls persist per workstation, rather than per workspace.

22. Must **REQ-UI-22 – Connectivity loss handling.** On loss of connectivity to Serval, scan engine, or detector, the UI shall display status and attempt reconnect every 1 s until success.

23. Must **REQ-UI-23 – Startup without connectivity.** If no connectivity is available at startup, the UI shall present a non-blocking status and allow offline configuration and analysis tools. Any setting or control relating to the unconnected device or server shall be blocked when not connected.

24. Must **REQ-UI-24 – Logging.** The UI shall write a log of important events to disk and to console; log level is controlled by environment variable `LOG_LEVEL`. Log location is user-configurable under `File > Preferences` but defaults to within the workspace configuration.

25. Must **REQ-UI-25 – Disk space guard (preflight + warning).** On *Start (Acquire)*, the system shall evaluate free space on the selected target volume. If free space is at or below the near-full threshold (default 5% of the volume), the system shall present a low-space warning before starting. If the volume reports 0 writable bytes free, the acquisition shall not start.

26. Must **REQ-UI-26 – Detector info and health available.** UI shall display from view ¿ detector health a UI panel describing the detectors information from serval

## 3.2 Capabilities (Non-Functional Requirements)

**Data Integrity**: Raw data measured using our device is sacred. We should do our utmost to ensure we collect all data requested by the user as accurately as possible. By default, the data shall not be modified in any way storage. This philosophy is shared with serval; output should be exactly as recorded, regardless of known problems in the hardware that need correction, including the tramlane effect and the VCO correction.

**Performance**: There are some performance-critical operations proposed in this software. Namely, the live processing needs to be able to keep up with the data acquisition and for adequate user feedback for the microscopist we need to minimize latency. Offline processing does not need the same performance requirement and just needs to run 'as fast as reasonable'.

**Scalability in Performance**: In the context of processing data from timepix detectors, scalability means the ability to handle more data at once. The processing requirements scale with the detector type and

layout. The tpx3 single chip can read out at a rate of 40Mbits/s while the 'turbo' edition can operate at 80Mbits/s. There are single, quad and 2x4 variations in the layout of detectors. Performance requirements scale linearly with the addition of every chip. So respectively, the 1, 4, and 8 chip detectors can read out a maximum of 80, 320 and 640Mbits/s and half this for the non-turbo versions. A single tpx4 device can read out a maximum of 160Gbits/s, which presents a challenge on an entirely new scale for data processing. Yet despite the engineering challenges, it is imperative that we build for the future generation of timepix as well as focusing on the now.

**Scalability in Memroy Usage**: Datasets are often large. Any offline processing must be able to handle data which are larger than what will fit in RAM. Online processing should not use more memory than necessary for processing.

## 3.3 Legal and Regulatory Requirements

Licenses. What can we do? What can't we do?